Rockchip Developer Guide Linux PCIE EP

文件标识：RK-KF-YF-C51

发布版本：V3.4.0

日期：2026-04-17

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档对RK PCIE EP Linux驱动开发使用进行说明。

产品版本

芯片名称	内核版本
RK3588	Linux 6.1、5.10
RK3568	Linux 6.1、5.10、4.19

说明：

- 仅在 RK PCIe EP 侧实现 RT-Thread 支持

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	yp.xiao，Kever Yang，Jon Lin	2023-03-07	初始版本
V1.1.0	yp.xiao	2023-05-22	更新SDK API以及部分配置说明
V1.2.0	yp.xiao	2023-06-15	添加EP卡固件编译、烧录、OTA升级说明 添加应用异常说明 更新`struct pcie_dev_st`结构体，支持超时配置、PCIE速率获取、RC DMA配置。
v1.3.0	Jon Lin	2023-10-07	增加pcie.bin说明
v1.4.0	yp.xiao, Jon Lin	2023-10-23	更新SDK API 添加EP验收标准章节 更新双存储说明 更新PCIE BIN说明
v1.5.0	Jon Lin	2024-05-23	更新PHY Mode设置
v1.6.0	Jon Lin	2024-09-18	添加用户态测试Demo - pcie_speed_test性能参考、增加BAR1/5支持
v1.7.0	yp.xiao, Jon Lin	2025-01-24	chips_connect开发包增加OSAL、兼容Windows版本
v1.8.0	Jon Lin	2025-02-12	优化EP启动方案说明、修改排版问题
v2.0.0	Jon Lin	2025-03-09	增加RK芯片互联独立供电方案说明
v3.0.0	Jon Lin	2025-05-14	调整须知为EP卡规格、优化EP Boot说明、增加数据传输原理说明、增加用户层多应用说明
v3.1.0	Jon Lin	2025-07-12	添加 RK1820
v3.2.0	Jon Lin	2025-08-11	独立输出PCIe EP Linux方案配置文档
v3.3.0	Jon Lin	2026-02-01	删除多余信息
v3.4.0	Jon Lin	2026-04-17	完善复位、休眠唤醒方案说明

目录

Rockchip Developer Guide Linux PCIE EP

1. 启动方案
    - 1.1 启动方案汇总
    - 1.2 PCIe Boot
    - 1.3 Flash Boot - 集成 PCIe Bin 及 EP Boot 软件方案
    - 1.4 Flash Boot - 集成 PCIe Bin 软件方案
    - 1.5 Common Boot
2. 硬件配置
    - 2.1 基础硬件设计
    - 2.2 **Warm Reset** 硬件设计补充说明
3. DTS配置
    - 3.1 **Warm Reset** 软件配置补充说明
4. Menuconfig配置
5. 设备驱动配置
6. 其他软件开发配置
    - 6.1 软件开发须知
    - 6.2 PCIe Bin阶段配置
        - 6.2.1 动态配置pcie.bin
        - 6.2.2 源码编译pcie.bin
            - 6.2.2.1 BSP配置
            - 6.2.2.2 BAR默认配置
            - 6.2.2.3 BAR大小配置
            - 6.2.2.4 BAR映射配置
            - 6.2.2.5 PHY模式配置
        - 6.2.3 PCIe Bin打包烧写
            - 6.2.3.1 工具烧写
            - 6.2.3.2 烧录器烧写
    - 6.3 SPL阶段配置
        - 6.3.1 Kconfig配置
        - 6.3.2 BAR默认配置
        - 6.3.3 BAR大小配置
        - 6.3.4 BAR映射配置
        - 6.3.5 PHY模式配置
        - 6.3.6 EP Boot配置
        - 6.3.7 SRNS配置
    - 6.4 Kernel阶段配置
        - 6.4.1 BAR大小配置
        - 6.4.2 BAR映射配置
        - 6.4.3 SRNS配置
7. 固件编译
8. 固件烧录
9. OTA
10. 常规复位方案（Conventional Reset）
    - 10.1 简介
    - 10.2 Cold Reset
    - 10.3 Warm Reset
    - 10.4 Hot Reset
11. 功能级复位说明（Function-Level Reset FLR）
12. 常见问题处理
    - 12.1 EP卡启动异常排查

1. 启动方案

1.1 启动方案汇总

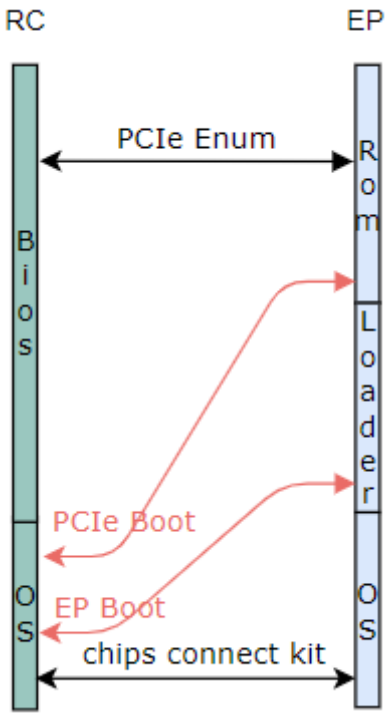
方案	优点	缺点
PCIe Boot	PCIe快速初始化、无存储、支持上位机下载固件	PCIe可调参数少、兼容性一般
Flash Boot - 集成PCIe Bin及EP Boot软件方案	支持通用RC、PCIe参数完全可调、小存储、支持上位机下载固件	软件流程相对复杂
Flash Boot - 集成PCIe Bin软件方案	支持通用RC、PCIe参数完全可调、支持本地存储	大存储
Common Boot	软件流程最简单、支持本地存储	大存储、仅支持特定RC

1.2 PCIe Boot

PCIe Boot为ROM code中集成的PCIe固件加载逻辑，PCIe Boot方案在ROM code中实现完成PCIe枚举、固件加载事务:

- PCIe快速初始化、无存储、支持上位机下载固件，需查阅"RK SOC PCIe互联特性"章节确认芯片支持情况
- 由于PCIe初始化代码为掩模固化代码，大多数参数不可调，仅支持OTP修改特定PCIe配置，推荐应用于固定类型的RC环境

以下为流程框图：



说明：

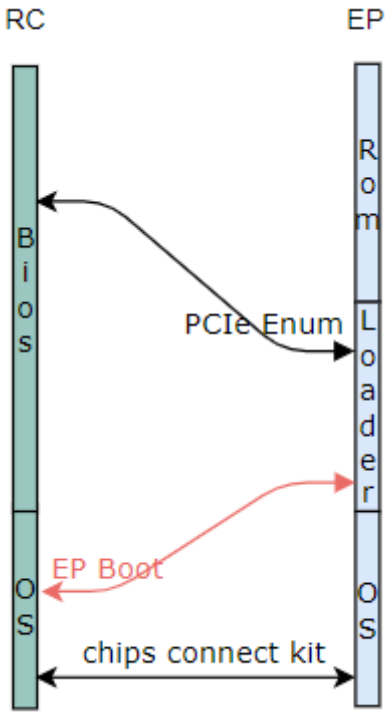
- 红色箭头涉及上位机下载固件行为
- 主要流程说明：
  - BootROM：完成PCIe枚举，集成PCIe Boot固件加载逻辑，上位机下载Pre-Loader固件，运行Pre-Loader固件
  - Loader：运行多个前级固件，通常包括：
    - ddr.bin：初始化ddr
    - loader.bin：集成EP Boot固件加载逻辑，上位机下载firmware固件，通常包括后级固件、算法等数据，运行后级固件，Linux方案该loader.bin为spl.bin
  - OS：后级固件，kernel/userspace下运行芯片互联 chips_connect 开发包

1.3 Flash Boot - 集成 PCIe Bin 及 EP Boot 软件方案

Flash Boot为RK SOC SPI Flash启动方案，Flash Boot方案在SPI Flash Boot基础上，于Loader阶段实现：

- 集成PCIe Bin固件实现PCIe快速初始化
- Loader阶段集成EP Boot逻辑，支持上位机推送后级固件

以下为流程框图：



须知：

- 使用SPI Flash作为启动存储，集成PCIe Bin固件实现PCIe快速初始化，支持由上位机推送后级固件
- 支持接入通用PC

说明：

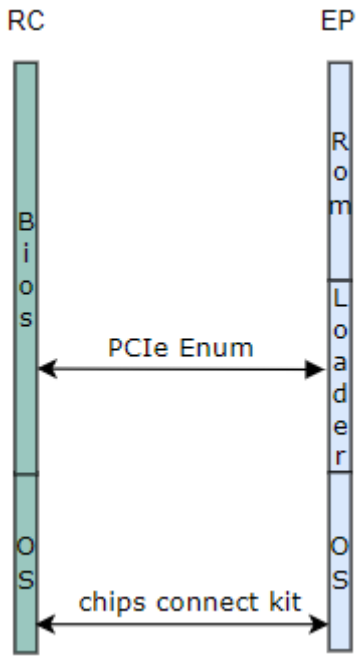
- 红色箭头为上位机引导固件下载
- 主要流程说明：
  - BootROM：加载运行PCIe Bin/Pre-Loader
  - PCIe Bin：完成PCIe枚举
  - Loader：运行多个前级固件，通常包括：
    - pcie.bin：完成PCIe枚举
    - ddr.bin：初始化ddr
  - loader.bin：集成EP Boot固件加载逻辑，上位机下载 fw packet 固件，通常包括后级固件、算法等数据，运行后级固件，Linux方案该loader.bin为spl.bin
  - OS：后级固件，kernel/userspace下运行芯片互联 chips_connect 开发包

1.4 Flash Boot - 集成 PCIe Bin 软件方案

Flash Boot为RK SOC SPI Flash启动方案，Flash Boot方案在SPI Flash Boot基础上，于Loader阶段实现：

- 集成 PCIe Bin 固件实现 PCIe 快速初始化，支持接入通用 PC
- 支持本地存储：
  - 单存储方案：SPI Flash 中加载后级固件
  - 双存储方案：SPI Flash 作为启动存储，并集成 NVMe/eMMC 等大容量存储组成双存储，从大容量存储中加载后级固件，详细参考《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》文档

以下为流程框图：



说明：

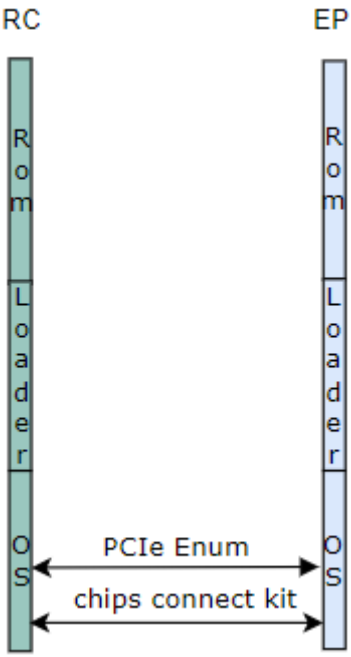
- 红色箭头为上位机引导固件下载
- 主要流程说明：
  - BootROM：加载运行PCIe Bin/Pre-Loader
  - PCIe Bin：完成PCIe枚举
  - Loader：运行多个前级固件，通常包括：
    - pcie.bin：完成PCIe枚举
    - ddr.bin：初始化ddr
  - loader.bin：加载运行后级固件
  - OS：后级固件，kernel/userspace下运行芯片互联 chips_connect 开发包
- 支持扩展双存储方案，SPI Flash启动存储，NVMe/eMMC等大容量存储作为主存储，详细参考《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》文档
- 该方案支持扩展为 SPI Nand Flash Boot

1.5 Common Boot

Common Boot 指 RK SOC 通用启动方案，支持：

- 内核阶段完成 PCIe 初始化及枚举
- 该方案由于未实现 PCIe 快速初始化，所以不支持对接通用 PC，适用于 RK 芯片互联、EP 板卡非标准独立供电方案、可定制 RC 主板等场景

以下为流程框图：



2. 硬件配置

2.1 基础硬件设计

详细的硬件设计可以向RK申请获取原理图参考设计，例如RK3588 <RK_PCIeEP_DEMO_RK3588_LP4XD200P232SD8_V10_20230323RZF.pdf>。PCIe EP产品，还应根据实际产品形态进一步调整硬件设计。

参考图纸默认配置下 EP 为 Cold Reset 常规复位方案，详细参考 “常规复位方案（Conventional Reset）” 章节介绍。

2.2 Warm Reset 硬件设计补充说明

EP 硬件要求：

- PERSTN 接入 EP GPIO，作为复位信号输入并注册为 GPIO 中断
- 理论上使用 3V3AUX 实现休眠过程中的供电，如需实现单独供电方案

RC 硬件要求：

- 推荐添加 RC PMU GPIO（输入）接 EP WAKEN 输出，以实现 EP 通知唤醒 RC 功能
- 推荐添加 RC GPIO 控制 EP PERSTN 信号，兜底复位流程
- 理论上提供 3V3AUX 休眠过程中为 EP 供电，如需实现单独供电方案

3. DTS配置

```
/{
    reserved-memory {
```

```
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;
        bar0_region: bar0-region@3c000000 {
            reg = <0x0 0x3c000000 0x0 0x00400000>;
        };
        bar2_region: bar2-region@40000000 {
            reg = <0x0 0x40000000 0x0 0x04000000>; # Bar大小配置：0x04000000Bytes
        };
    };
}

&pcie3x4 {
        compatible = "rockchip,rk3588-pcie-std-ep";
        memory-region = <&bar0_region>, <&bar2_region>;
        memory-region-names = "bar0", "bar2";
        reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_LOW>;
        rockchip,ep-power-independent;
        status = "okay";
};
```

3.1 Warm Reset 软件配置补充说明

如 RK EP PCIe 改用 Warm Reset 复位方案，使用 PERSTN/WAKEN GPIO 作为复位同步信号，需添加以下配置：

1. `reset-gpios = <&gpio3 13 GPIO_ACTIVE_HIGH>;`

必须配置项: 此项用于设置 PCIe 接口的 PERSTN 复位输入信号；不论是插槽式还是焊贴式设备，请在原理图上找到该引脚，并正确配置。否则可能无法稳定完成链路建立。

2. `wake-gpios = <&gpio3 13 GPIO_ACTIVE_HIGH>;`

必须配置项: 此项用于设置 PCIe 接口的 WAKEN 输出信号；不论是插槽式还是焊贴式设备，请在原理图上找到该引脚，并正确配置。否则无法实现休眠唤醒向主机发送信号通知。

3. `rockchip,ep-power-independent;`

可选配置项：此项用于设置 PCIe 上电过程中添加 PERSTN 信号同步信息，标准流程默认 PERSTN 已处于释放状态，无需配置，但对于 **EP 供电独立控制方案** 需要开启此项配置。

4. Menuconfig配置

```
CONFIG_PCIE_DW_ROCKCHIP_EP=y
# CONFIG_STRICT_DEVMEM is not set    # 建议开发阶段关闭 STRICT_DEVMEM 限制，方便使用 io 命令调试
```

5. 设备驱动配置

PCIe EP（Endpoint）设备驱动是运行在Root Complex（RC）侧的驱动程序，用于控制总线上的数据传输，并为应用层提供所需的内核接口。

RK RC Kconfig配置

```
CONFIG_PCIE_FUNC_RKEP=y
```

通用RC-FUNC驱动

RK Chips Connect 开发包支持 KO 编译，详细参考 `chips_connect/patch-kernelx.xx/通用 RC-FUNC 驱动/`。

6. 其他软件开发配置

6.1 软件开发须知

EP 卡部分软件配置为关联配置，如做修改应修改所有关联处，如关于 Bar 大小配置：

- PCIe Bin阶段-Bar大小配置（双存储方案）
- SPL阶段-Bar大小配置

6.2 PCIe Bin阶段配置

PCIe bin仅支持完成PCIe控制器EP mode功能的初始化，且早于dram.bin初始化。

提供编译完成的 pcie.bin 以及源码供自定义修改，pcie.bin 存放在 SDK/rkbin/bin/rkxx/对应文件下，源码路径如下：`patch-*/bootloader`

建议客户直接修改 pcie.bin 配置，除非特殊需求再通过源码自定义编译。

6.2.1 动态配置pcie.bin

pcie.bin支持直接修改bin文件中预留的配置空间从而实现常用的PCIe配置变换，目前已经实现以下配置修改：

- PCIe Generation
- PCIe lanes
- pcie.bin串口波特率
- PCIe vendor id
- PCIe device id

修改用脚本为 rkbin/tools/pcie_idb_config.sh，命令介绍：

```
./tools/pcie_idb_config.sh <soc> <gen> <lanes> <uart_baud_rate> <vid_l> <vid_h>
<did_l> <did_h> <input>
like following:
./tools/pcie_idb_config.sh RK3588 3 4 1500000 87 1d 6a 35
bin/rk35/rk3588_pcie_v*.bin #vid=0x1d87 did=0x356a
./tools/pcie_idb_config.sh RK3588 3 2 1500000 87 1d 6a 35
bin/rk35/rk3588_pcie_v*.bin
./tools/pcie_idb_config.sh RK3588 3 1 1500000 87 1d 6a 35
bin/rk35/rk3588_pcie_v*.bin
./tools/pcie_idb_config.sh RK3588 3 4 115200  87 1d 6a 35
bin/rk35/rk3568_pcie_v*.bin
./tools/pcie_idb_config.sh RK3568 3 2 1500000 87 1d 6a 35
bin/rk35/rk3588_pcie_v*.bin
```

若以上开放配置不满足需求，客户可以修改源码自定义其他功能。

6.2.2 源码编译pcie.bin

源码依赖 rkbin 内的工具，要求源码解压在 rkbin/ 同级目录进行开发，或者直接修改 tools/mk_pcie_idb.sh 里 boot_merger 和 programmer_image_tool 工具路径

编译工具链

从src/platform/common/GCC/Cortex-A.mk中确认工具链：

```
################################################################################
# Cross compiler
################################################################################
ifneq ($(wildcard ${ROOT_PATH}/../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.2-
2020.11-x86_64-aarch64-none-linux-gnu/bin),)
CROSS_COMPILE   ?= ${ROOT_PATH}/../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.2-
2020.11-x86_64-aarch64-none-linux-bin/aarch64-none-linux-gnu-
else ifeq ($(wildcard /opt/prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-
11.3.rel1-x86_64-aarch64-none-linux-gnu/bin),)
CROSS_COMPILE   ?= /opt/prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-
11.3.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
else
CROSS_COMPILE   ?= aarch64-none-linux-gnu-
endif
```

编译命令

RK3588：

```
./tools/mk_pcie_idb.sh RK3588
```

RK3568：

```
./tools/mk_pcie_idb.sh RK3568
```

输出文件：

pcie_idb.img

6.2.2.1 BSP配置

以RK3588为例，源码src/platform/RK3588/bsp/platform.h

```
#define BOOT_USING_UART_BAUD 1500000                    // 修改波特率，支持
1500000或115200
```

6.2.2.2 BAR默认配置

源码src/driver/pcie/pcie_ep.c

Demo使用如下默认配置，除了BAR4，项目可以根据实际需求进行修改：

- BAR0： 32bits 4MB，命令流；
- BAR2： 64bits 64MB，数据流；
- BAR4：32bits 1MB，EP端PCIe寄存器，不可修改；

其中BAR4的offset映射如下，可以通过RC端直接操作对应寄存器：

- 0: DMA
- 0x2000: iATU
- 0x20000: MSI-X table

6.2.2.3 BAR大小配置

在PCIe_EP_Init()完成BAR的size配置，通过控制器的resize BAR功能实现。

配置BAR SIZE，(bar_id * 0x8)指定目标Bar ID，bit[13:8]用于配置BAR size，结果为2^n，所以支持1MB(2^0)到8EB(2^20)，以下为Bar0~5配置：

```
    writel(0x2c0, resbar_base + 0x8 + 0 * 0x8);
    writel(0x2c0, resbar_base + 0x8 + 1 * 0x8);
    writel(0x6c0, resbar_base + 0x8 + 2 * 0x8);
    writel(0x6c0, resbar_base + 0x8 + 3 * 0x8);
    writel(0x0c0, resbar_base + 0x8 + 4 * 0x8); /* wired register */
    writel(0x6c0, resbar_base + 0x8 + 5 * 0x8);
```

配置BAR属性，选择已定义的宏进行配置，支持32bit和64bit，是否PREFETCH：

```
    rockchip_pcie_ep_set_bar_flag(dbi_base, 0, PCI_BASE_ADDRESS_MEM_TYPE_32);
    rockchip_pcie_ep_set_bar_flag(dbi_base, 2, PCI_BASE_ADDRESS_MEM_PREFETCH |
PCI_BASE_ADDRESS_MEM_TYPE_64);
    rockchip_pcie_ep_set_bar_flag(dbi_base, 4, PCI_BASE_ADDRESS_MEM_TYPE_32);
```

关掉不使用的BAR：

```
    //writel(0x0, dbi_base + 0x100000 + 0x10 + 0 * 4);
    writel(0x0, dbi_base + 0x100000 + 0x10 + 1 * 4); /* BAR1 disabled */
    //writel(0x0, dbi_base + 0x100000 + 0x10 + 2 * 4);
    //writel(0x0, dbi_base + 0x100000 + 0x10 + 3 * 4);
    //writel(0x0, dbi_base + 0x100000 + 0x10 + 4 * 4);
    writel(0x0, dbi_base + 0x100000 + 0x10 + 5 * 4);  /* BAR5 disabled */
```

须知：

- 映射PCIe Bin阶段-Bar大小配置 为关联配置，应同时修改：
  - PCIe Bin阶段-Bar大小配置（双存储方案）
  - SPL阶段-Bar大小配置
  - Kernel阶段-Bar大小配置

6.2.2.4 BAR映射配置

以 RK3588 为例，源码 src/platform/RK3588/bsp/platform.h

```
#define BOOT_USING_PCIE_EP_BAR0_CPU_ADDRESS 0x3C000000    // 调整Bar0映射地址
```

代码中实现：

```
HAL_PCIE_InboundConfig(pcie, 0, 0, BOOT_USING_PCIE_EP_BAR0_CPU_ADDRESS);
```

须知：

- PCIe.bin阶段-Bar映射配置与Kernel阶段配置独立，可根据需求映射不同空间

6.2.2.5 PHY模式配置

RK3588 默认 phy0/1 为合并使用以支持 Gen3x4，也可拆分为 Gen3x2 或 Gen3x1 模式，配置如下：

RK3588 PCIe PHY 3x2实例：

```
diff --git a/src/hal/lib/bsp/RK3588/hal_bsp.c b/src/hal/lib/bsp/RK3588/hal_bsp.c
index 4a6c2e6..f03995b 100644
--- a/src/hal/lib/bsp/RK3588/hal_bsp.c
+++ b/src/hal/lib/bsp/RK3588/hal_bsp.c
@@ -123,7 +123,7 @@ const struct HAL_UART_DEV g_uart9Dev =
 #ifdef HAL_PCIE_MODULE_ENABLED
 const struct HAL_PHY_SNPS_PCIE3_DEV g_phy_pcie3Dev =
 {
-    .phyMode = PHY_MODE_PCIE_AGGREGATION,
+    .phyMode = PHY_MODE_PCIE_NANBNB,
 };

 struct HAL_PCIE_DEV g_pcieDev =
```

RK3588 PCIe PHY 3x1实例：

```
diff --git a/src/hal/lib/bsp/RK3588/hal_bsp.c b/src/hal/lib/bsp/RK3588/hal_bsp.c
index 4a6c2e6..f03995b 100644
--- a/src/hal/lib/bsp/RK3588/hal_bsp.c
+++ b/src/hal/lib/bsp/RK3588/hal_bsp.c
@@ -123,7 +123,7 @@ const struct HAL_UART_DEV g_uart9Dev =
 #ifdef HAL_PCIE_MODULE_ENABLED
 const struct HAL_PHY_SNPS_PCIE3_DEV g_phy_pcie3Dev =
 {
-    .phyMode = PHY_MODE_PCIE_AGGREGATION,
+    .phyMode = PHY_MODE_PCIE_NABIBI,
 };

 struct HAL_PCIE_DEV g_pcieDev =
```

6.2.3 PCIe Bin打包烧写

6.2.3.1 工具烧写

1. pcie.bin转pcie_idb.img

```
#生成 loader
./tools/boot_merger RKBOOT/RK3588MINIALL_PCIE_EP.ini
$git:(master) $ ls rk3588_pcie_loader_v1.00.bin
rk3588_pcie_loader_v1.00.bin

#spinand 制作 idb，要求使用 spinand type 做对齐
./tools/programmer_image_tool -i rk3588_pcie_loader_v1.00.bin -b 128 -p 2 -t
spinand -o ./
#emmc
./tools/programmer_image_tool -i rk3588_pcie_loader_v1.00.bin -t emmc -o ./

#制作双备份的 pcie_idb.img
cat idblock.img > pcie_idb.img
cat idblock.img >> pcie_idb.img
rm idblock.img
```

2. 分立烧写
   - 工具添加pcie_idb分区，emmc flash起始地址：0x40；spi flash起始地址：0x100;
   - 修改烧写工具配置文件，烧写工具目录下config.ini使能配置：`CLOSE_CHECK_IDB=TRUE`
   - 板子进入Maskrom模式，工具点击烧写
3. 完整包烧写
   - 修改分区表添加pcie_idb分区，如下是emmc分区，spi nand只是地址不同

```
CMDLINE:
mtdparts=rk29xxnand:0x00000400@0x00000040(pcie_idb),0x00000800@0x00002000(se
curity),0x00002000@0x00004000(uboot),0x00010000@0x00006000(boot),0x00064000@
0x00016000(rootfs),-@0x0007a000(userdata:grow)
```

   - 修改package-file，tools/linux/Linux_Pack_Firmware/rockdev/rk3588-package-file-nvr-emmc / spi-nand

```
package-file    package-file
bootloader      Image/MiniLoaderAll.bin
pcie_idb        Image/pcie_idb.img    #添加pcie_idb文件
parameter       Image/parameter.txt
uboot           Image/uboot.img
boot            Image/boot.img
rootfs          Image/rootfs.img
userdata        Image/userdata.img
```

- 执行build_emmc.sh 编译update.img烧写

6.2.3.2 烧录器烧写

参考EP卡固件烧录

6.3 SPL阶段配置

6.3.1 Kconfig配置

本功能在SPL中有以下三个Kconfig项，对于需要EP Boot功能的产品需要全部选上，对于不需要EP Boot功能的产品只需要选上SPL_PCIE_EP_SUPPORT即可。

```
CONFIG_SPL_PCIE_EP_SUPPORT=y
CONFIG_SPL_RAM_SUPPORT=y
CONFIG_SPL_RAM_DEVICE=y
```

6.3.2 BAR默认配置

Demo使用如下默认配置，除了BAR4，项目可以根据实际需求进行修改：

- BAR0： 32bits 4MB，命令流；
- BAR2： 64bits 64MB，数据流；
- BAR4：32bits 1MB，EP端PCIe寄存器，不可修改；

其中BAR4的offset映射如下，可以通过RC端直接操作对应寄存器：

- 0: DMA
- 0x2000: iATU
- 0x20000: MSI-X table

6.3.3 BAR大小配置

在pcie_bar_init()完成BAR的size配置，通过控制器的resize BAR功能实现。

配置BAR SIZE，offset+0x8寄存器，bit[13:8]用于配置BAR size，结果为2^n，所以支持1MB(2^0)到8EB(2^20)：

```
        /* Resize BAR0 to support 4M 32bits */
        resbar_base = dbi_base + 0x2e8;
        writel(0xffffff0, resbar_base + 0x4);
        writel(0x2c0, resbar_base + 0x8);
        /* BAR2: 64M 64bits */
        writel(0xffffff0, resbar_base + 0x14);
        writel(0x6c0, resbar_base + 0x18);
        /* BAR4: Fixed for EP wired register, 1M 32bits */
        writel(0xffffff0, resbar_base + 0x24);
        writel(0xc0, resbar_base + 0x28);
```

配置BAR属性，选择已定义的宏进行配置，支持32bit和64bit，是否PREFETCH：

```
        rockchip_pcie_ep_set_bar_flag(dbi_base, 0,
PCI_BASE_ADDRESS_MEM_TYPE_32);
        rockchip_pcie_ep_set_bar_flag(dbi_base, 2, PCI_BASE_ADDRESS_MEM_PREFETCH
| PCI_BASE_ADDRESS_MEM_TYPE_64);
        rockchip_pcie_ep_set_bar_flag(dbi_base, 4,
PCI_BASE_ADDRESS_MEM_TYPE_32);
```

关掉不使用的BAR：

```
        /* Close bar1 bar3 bar5 */
        writel(0x0, dbi_base + 0x100000 + 0x14);
        //writel(0x0, dbi_base + 0x100000 + 0x18);
        writel(0x0, dbi_base + 0x100000 + 0x1c);
        //writel(0x0, dbi_base + 0x100000 + 0x20);
        writel(0x0, dbi_base + 0x100000 + 0x24);
        /* Close ROM BAR */
        writel(0x0, dbi_base + 0x100000 + 0x30);
```

须知：

- SPL 阶段-Bar大小配置为关联配置，应同时修改：
  - PCIe Bin阶段-Bar大小配置（双存储方案）
  - SPL阶段-Bar大小配置
  - Kernel阶段-Bar大小配置

6.3.4 BAR映射配置

每个BAR对应的空间，通过pcie_inbound_config()配置映射到内部的内存地址区域，可以修改RKEP_BAR0_ADDR， RKEP_BAR2_ADDR的宏定义来修改要映射的目标地址。

```
        /* BAR0: RKEP_BAR0_ADDR */
        writel(RKEP_BAR0_ADDR, base + PCIE_ATU_CPU_ADDR_LOW);
        writel(0, base + PCIE_ATU_CPU_ADDR_HIGH);
        writel(0, base + PCIE_ATU_UNR_REGION_CTRL1);
        /* PCIE_ATU_UNR_REGION_CTRL2 */
        writel(PCIE_ATU_ENABLE | PCIE_ATU_BAR_MODE_ENABLE | (0 << 8),
               base + PCIE_ATU_UNR_REGION_CTRL2);
```

须知：

- SPL阶段-Bar映射配置与Kernel阶段配置独立，可根据需求映射不同空间

6.3.5 PHY模式配置

Demo默认使用的是0xfe150000控制器PCIe3x4模式，也就是PHY也是4lane模式，由于PHY是支持拆分的，如果方案需要修改为其他模式，可以修改pcie_cru_init()的对应配置, 如PCIe工作于2lane+2lane模式可以将PHY_MODE_PCIE_AGGREGATION修改为PHY_MODE_PCIE_NANBNB或其他：

RK3588 PCIe PHY 3x2实例：

```
diff --git a/arch/arm/mach-rockchip/spl_pcie_ep_boot.c b/arch/arm/mach-
rockchip/spl_pcie_ep_boot.c
index 8b054d91558..54f9d22cde2 100644
--- a/arch/arm/mach-rockchip/spl_pcie_ep_boot.c
+++ b/arch/arm/mach-rockchip/spl_pcie_ep_boot.c
@@ -384,7 +384,7 @@ static void pcie_board_init(void)
 #define PHY_MODE_PCIE_NABINB   2       /* P1:PCIe3x1*2 + P0:PCIe3x2 */
 #define PHY_MODE_PCIE_NABIBI   3       /* P1:PCIe3x1*2 + P0:PCIe3x1*2 */

-#define PHY_MODE_PCIE                  PHY_MODE_PCIE_AGGREGATION
+#define PHY_MODE_PCIE                  PHY_MODE_PCIE_NANBNB

 #define CRU_BASE_ADDR                  0xfd7c0000
 #define CRU_SOFTRST_CON32              (CRU_BASE_ADDR + 0x0a80)
```

RK3588 PCIe PHY 3x1实例：

```
diff --git a/arch/arm/mach-rockchip/spl_pcie_ep_boot.c b/arch/arm/mach-
rockchip/spl_pcie_ep_boot.c
index 8b054d91558..54f9d22cde2 100644
--- a/arch/arm/mach-rockchip/spl_pcie_ep_boot.c
+++ b/arch/arm/mach-rockchip/spl_pcie_ep_boot.c
@@ -384,7 +384,7 @@ static void pcie_board_init(void)
 #define PHY_MODE_PCIE_NABINB   2       /* P1:PCIe3x1*2 + P0:PCIe3x2 */
 #define PHY_MODE_PCIE_NABIBI   3       /* P1:PCIe3x1*2 + P0:PCIe3x1*2 */

-#define PHY_MODE_PCIE                  PHY_MODE_PCIE_AGGREGATION
+#define PHY_MODE_PCIE                  PHY_MODE_PCIE_NABIBI

 #define CRU_BASE_ADDR                  0xfd7c0000
 #define CRU_SOFTRST_CON32              (CRU_BASE_ADDR + 0x0a80)
```

6.3.6 EP Boot配置

原理介绍中提到EP会等待固件并运行固件，涉及跟RC间的固件下载协议和BootLoader前后级的固件位置约定。

固件的下载函数pcie_wait_for_fw()和后级固件位置约定函数pcie_update_atags(), 可以通过下文提到的地址来做对应修改，请务必保证RC与EP约定的命令和地址一致，BootLoader前后级约定的固件位置一致，才能顺利完成整个固件的下载和启动过程。如果不理解全过程，请勿修改。

跟RC间的固件下载协议内容是，先按要求把固件下载到指定位置，然后发送运行固件命令。

1. 固件下载位置

约定下载以下两个固件:

固件名	打包格式	内容	下载地址
uboot.img	FIT	ATF和U-Boot	BAR2+0
boot.img	FIT	Kernel、DTB、ramdisk	BAR2+0x400000

2. 运行固件命令

命令名称	命令位置	命令值	操作
CMD_LOADER_RUN	BAR0 + 0x400	0x524b4501	运行下载的固件

从RC下载过来的两个固件，其中uboot.img会被SPL(运行EP Boot的环境)直接解析并运行，然后通过一个RK自定义的RAM_PARTITION来传递boot.img的位置给U-Boot。pcie_update_atags()函数定义了boot分区在RAM的起始地址和SIZE，由于默认使用的BAR2是64M，其中起始4M分给了uboot.img，后续的60M分给了boot.img，这是为了方便RC端直接通过CPU从BAR2把固件写过来，如果size不够，可以在RC端调用EP的dma来下载固件，这样size不受BAR2大小限制，需要注意同步更新RAM_PARTITION中的size配置。

6.3.7 SRNS配置

对于可以使用SRNS模式参考时钟的方案，需要打开相关配置，打开配置后EP端会使用内部的100MHz时钟作为参考时钟，不依赖外部提供的时钟。在代码中直接打开如下宏配置即可：

```
/* SRNS: Use Seperate refclk(internal clock) instead of from RC */
// #define PCIE_ENABLE_SRNS_PLL_REFCLK
```

须知：

- SPL 阶段-SRNS配置为关联配置，应同时修改：
  - PCIe Bin阶段-SRNS配置（双存储方案）
  - SPL阶段-SRNS大小配置
  - Kernel阶段-SRNS配置

6.4 Kernel阶段配置

6.4.1 BAR大小配置

在rockchip_pcie_resize_bar完成BAR的size配置，通过控制器的resize BAR功能实现。

配置BAR SIZE，offset+0x8寄存器，bit[13:8]用于配置BAR size，结果为2^n，所以支持1MB(2^0)到8EB(2^20)，配置BAR属性，选择已定义的宏进行配置，支持32bit和64bit，是否PREFETCH：

```
/* Resize BAR0 4M 32bits, BAR2 64M 64bits-pref, BAR4 1MB 32bits */
bar = BAR_0;
dw_pcie_writel_dbi(pci, resbar_base + 0x4 + bar * 0x8, 0xfffff0);
```

```
dw_pcie_writel_dbi(pci, resbar_base + 0x8 + bar * 0x8, 0x2c0);
rockchip_pcie_ep_set_bar_flag(rockchip, bar, PCI_BASE_ADDRESS_MEM_TYPE_32);

bar = BAR_2;
dw_pcie_writel_dbi(pci, resbar_base + 0x4 + bar * 0x8, 0xfffff0);
dw_pcie_writel_dbi(pci, resbar_base + 0x8 + bar * 0x8, 0x6c0);
rockchip_pcie_ep_set_bar_flag(rockchip, bar,PCI_BASE_ADDRESS_MEM_PREFETCH |
PCI_BASE_ADDRESS_MEM_TYPE_64);

bar = BAR_4;
dw_pcie_writel_dbi(pci, resbar_base + 0x4 + bar * 0x8, 0xfffff0);
dw_pcie_writel_dbi(pci, resbar_base + 0x8 + bar * 0x8, 0xc0);
rockchip_pcie_ep_set_bar_flag(rockchip, bar, PCI_BASE_ADDRESS_MEM_TYPE_32);
```

关掉不使用的BAR：

```
/* Disable BAR1 BAR5*/
bar = BAR_1;
dw_pcie_writel_dbi(pci, PCIE_TYPE0_HDR_DBI2_OFFSET + 0x10 + bar * 4, 0);
bar = BAR_5;
dw_pcie_writel_dbi(pci, PCIE_TYPE0_HDR_DBI2_OFFSET + 0x10 + bar * 4, 0);
```

须知：

- Kernel阶段-Bar大小配置为关联配置，应同时修改：
  - PCIe Bin阶段-Bar大小配置（双存储方案）
  - SPL阶段-Bar大小配置
  - Kernel阶段-Bar大小配置
- Kernel阶段-Bar映射配置与SPL阶段配置独立，可根据需求映射不同空间

6.4.2 BAR映射配置

每个BAR对应的空间，通过配置映射到内部的内存地址区域，可以修改dts来修改要映射的目标地址。

```
reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;
        bar0_region: bar0-region@3c000000 {
                reg = <0x0 0x3c000000 0x0 0x00400000>;
        };
        bar2_region: bar2-region@40000000 {
                reg = <0x0 0x40000000 0x0 0x04000000>;
        };
};
```

说明：

- 支持新增bar1/bar5映射内存地址，同时自动使能bar1/5，如果使用PCIe Bin/SPL初始化PCIe，需参考“BAR大小配置”注释掉bar1/5 disabled代码

6.4.3 SRNS配置

SRNS配置为补丁形式添加，请从开发包中获取补丁文件 0001-TEST-kernel5.10-rk3588-SRNS.patch

Kernel 阶段-SRNS配置为关联配置，应同时修改：

- PCIe Bin阶段-SRNS配置（双存储方案）
- SPL阶段-SRNS大小配置
- Kernel阶段-SRNS配置

7. 固件编译

参考patch目录说明打上对应补丁后根据场景选择对应的编译说明：

Flash Boot - 集成PCIe Bin软件方案

单存储：

- 增加pcie_idb分区，升级pcie_idb.img镜像
- 增加normal_idb分区，升级normal_idb.img镜像，由MiniLoaderAll.bin自动升级
- 其余镜像同普通固件一样不需要其它特殊配置，直接编译出各个分区img以及update.img。

双存储，参考《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》文档“RK PCIe EP 双存储方案”相关章节说明。

- SDK默认不支持把双存储固件打包成update.img，需要借助tools目录下的dual_storage工具打包，参考tools/dual_storage/README.txt说明把各个分区img固件拷贝到对应目录下，执行build.sh即可把分立img打包成update.img。

Flash Boot - 集成PCIe Bin及EP Boot软件方案

对接通用PC：

- 增加pcie_idb分区，升级pcie_idb.img镜像
- 增加normal_idb分区，升级normal_idb.img镜像，由MiniLoaderAll.bin自动升级
- 其余固件由上位机推送

对接RK RC：

- 可省略PCIe Bin集成，仅SPL实现EP Boot功能，EP卡存储器件仅储存Miniloader镜像
- 其余固件由上位机推送

Common Boot

和普通固件一样不需要其它特殊配置，直接编译出各个分区img以及update.img。

8. 固件烧录

- 单存储

  单存储方案固件烧写与正常固件烧写方式一致，参考各芯片发布文档刷机说明章节即可。

- 双存储

  详细参考 《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》文档”RK PCIe EP 双存储方案固件打包“章节。

9. OTA

分区升级原理：通过分区表信息让内核创建对应的设备节点，上层通过写入/dev/下的设备节点实现分区更新。

OTA升级可以使用RK方案通过recovery模式进行分区更新，或者使用自定义方案，只需要写入/dev/下的设备节点。

1. Flash Boot - 集成PCIe Bin软件方案OTA

- 单存储

  单存储方案OTA和正常固件方式一致，参考 docs\Linux\Recovery\Rockchip_Developer_Guide_Linux_Recovery_CN.pdf章节更新即可。

- 双存储

  详细参考 《Rockchip_Developer_Guide_Dual_Storage_CN.pdf》文档”RK PCIe EP 双存储方案 OTA 升级 “章节。

2. Flash Boot - 集成PCIe Bin及EP Boot软件方案OTA

   EP Boot方案只有loader分区，需要在分区表中添加loader分区信息。

   Nor：可以参考EP卡OTA->双存储章节->Nor配置 Uboot分区配置。

   EMMC：可以参考EP卡OTA->双存储章节->EMMC配置，直接在分区表添加loader信息。

   RK OTA方案不支持单独升级loader，需要自定义实现。

   不推荐对loader分区进行升级，风险较高。

注意：MiniLoaderAll.bin和parameter.txt 不能直接写到/dev/下的设备节点，需要转成二进制文件才能写入。

10. 常规复位方案（Conventional Reset）

10.1 简介

PCIe 允许三种类型的复位方案，包括：

- Cold Reset，休眠唤醒过程 EP 下电，使用 PERSTN 信号控制复位同步
- Warm Reset，休眠唤醒过程 EP 保持供电，理论上使用 3V3AUX 供电，协议未详细定义其他具体流程
- hot reset，PCIe 带内复位机制

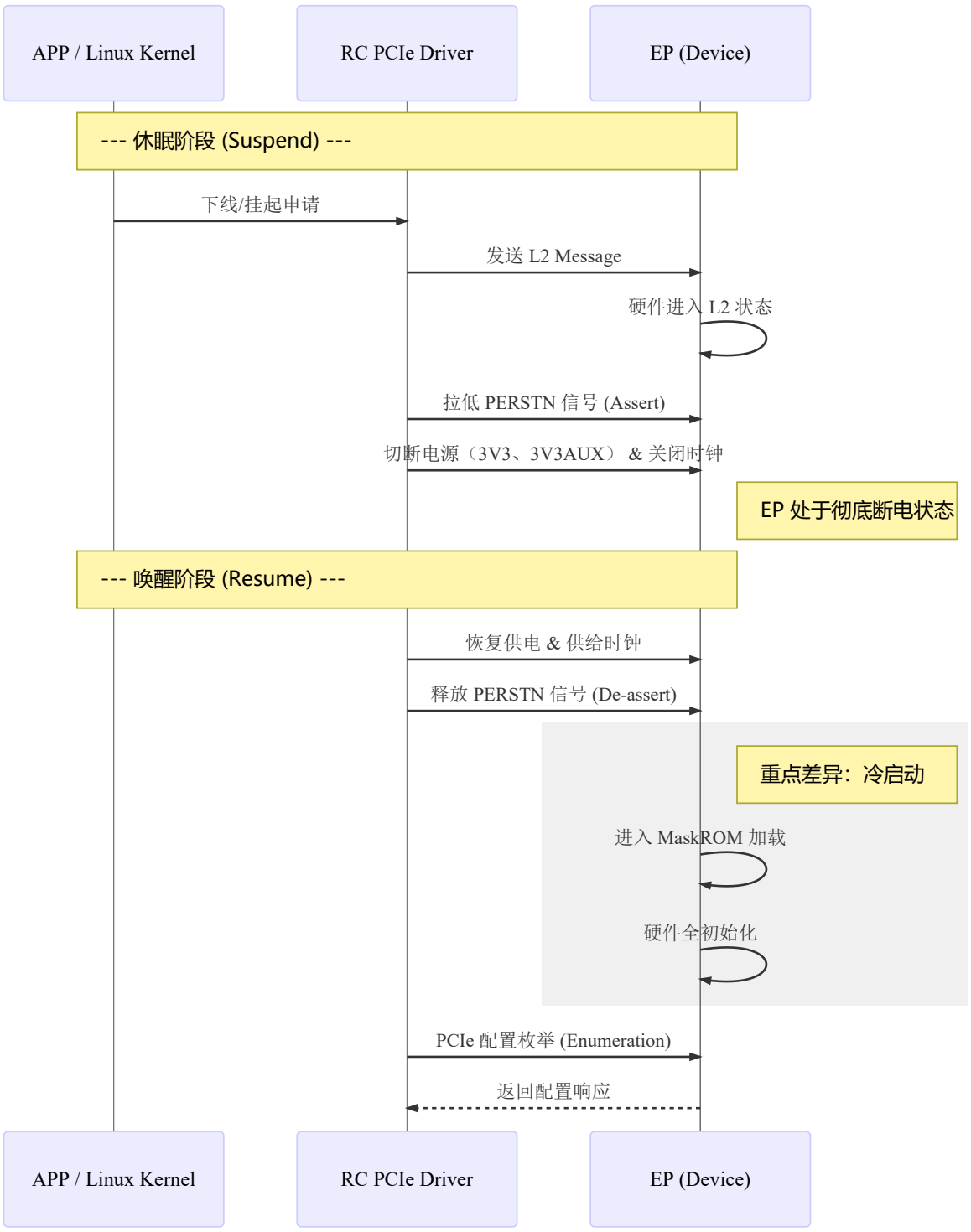
RK PCIe EP 大部分方案默认采用 Cold Reset 设计。

10.2 Cold Reset

特点

- 休眠过程 EP 下电，数据不保持，唤醒后系统完全复位
- PERSTN 支持复位系统，无需额外添加 GPIO 信号兜底 EP 复位流程

休眠唤醒流程



10.3 Warm Reset

特点

- 休眠过程 EP 保持供电，数据保持
- 需要 PERSTN、WAKEN 信号作为同步信号
- 推荐添加 RC GPIO 控制 EP PERSTN 信号，兜底复位流程

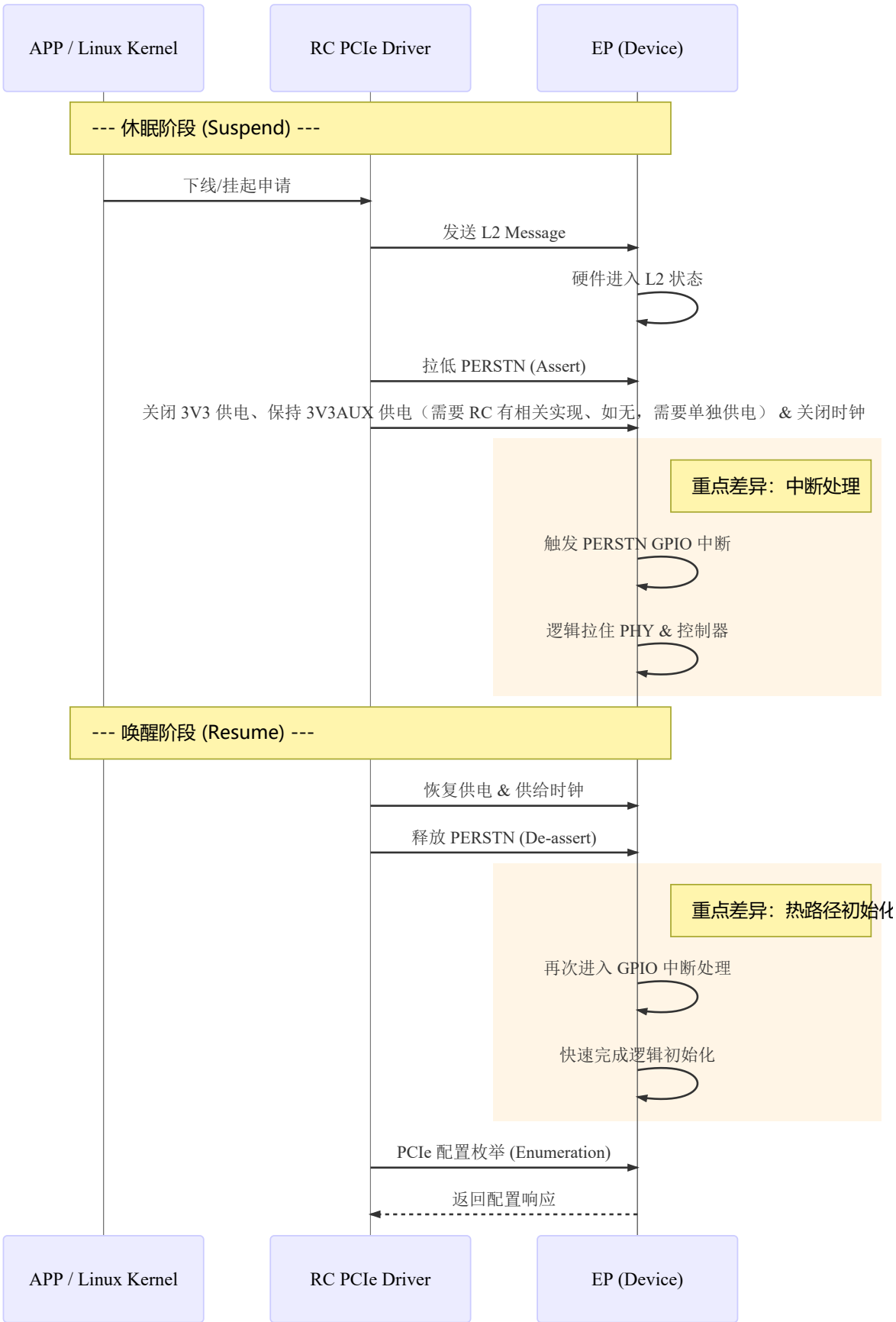
测试须知

- 该方案涉及较多软硬件定制化流程，需进行专项拷机测试
- 该方案为 RK 参考方案，用户可根据项目经验和业务需求进一步优化
- EP 端需支持独立运行能力，并在软件层面增加相应控制逻辑

常见问题

- 如为 RK 互联方案，且 EP 仅在内核阶段进行 PCIe 初始化，则 RK RC 需配置 `rockchip,wait-for-link-ms` 属性延长枚举等待时间

休眠唤醒流程


APP / Linux Kernel
RC PCIe Driver
EP (Device)
--- 休眠阶段 (Suspend) ---
下线/挂起申请
发送 L2 Message
硬件进入 L2 状态
拉低 PERSTN (Assert)
关闭 3V3 供电、保持 3V3AUX 供电（需要 RC 有相关实现、如无，需要单独供电） & 关闭时钟
重点差异：中断处理
触发 PERSTN GPIO 中断
逻辑拉住 PHY & 控制器
--- 唤醒阶段 (Resume) ---
恢复供电 & 供给时钟
释放 PERSTN (De-assert)
重点差异：热路径初始化
再次进入 GPIO 中断处理
快速完成逻辑初始化
PCIe 配置枚举 (Enumeration)
返回配置响应
APP / Linux Kernel
RC PCIe Driver
EP (Device)

10.4 Hot Reset

RK PCIe EP 支持该复位机制，RC 支持 Secondary Bus Reset 机制，配置并触发 PCIe 带内复位机制，它通过在 PCIe 链路上发送特定序列（带 Hot Reset 标志的 TS1），实现在不断电且无需操作物理引脚的情况下重启并初始化下游设备。

11. 功能级复位说明（Function-Level Reset FLR）

RK PCIe EP 无 Function-Level 复位需求，未实现该机制。FLR 机制允许系统软件以“功能 (Function)”为最小粒度，对 PCIe 端点 (Endpoint) 硬件进行静默（挂起）和复位，而不影响同一物理设备上的其他功能。

12. 常见问题处理

12.1 EP卡启动异常排查

EP卡SPL阶段打印

```
U-Boot SPL board init
RKEP: Init PCIe EP
RKEP: 175301 - PHY Mode 0x4
RKEP: 176542 - RefClock in common clock_mode
RKEP: 187578 - BAR0: 0x3c000000
RKEP: 187848 - BAR2: 0x010000000
RKEP: 188227 - init PCIe fast Link up
RKEP: 208576 - Link up 230011
RKEP: 243683 - Done
```

问题分析：

- 如设备仅运行到"RKEP: Init PCIe EP"，通常为枚举fail，请确认EP卡的参考时钟、12V供电是否正常，PERSTN 是否已释放

EP卡Kernel阶段打印

```
[root@RK3588:/]# dmesg | grep pci
[    3.371073] rk-pcie-ep fe150000.pcie: bar0: assigned [0x3c000000-3c3fffff]
[    3.371890] rk-pcie-ep fe150000.pcie: bar2: assigned [0x40000000-43ffffff]
[    3.371901] rk-pcie-ep fe150000.pcie: no vpcie3v3 regulator found
[    3.371918] rk-pcie-ep fe150000.pcie: already linkup
[    3.372001] rk-pcie-ep fe150000.pcie: register misc device pcie_ep
[    3.725930] ehci-pci: EHCI PCI platform driver
[    3.869983] pcie30_avdd1v8: supplied by avcc_1v8_s0
[    3.870541] pcie30_avdd0v75: supplied by avdd_0v75_s0
```

EP卡设备驱动打印

内核模块：

```
rk3588_s:/ # dmesg | grep rkep
[    2.278964] pcie-rkep 0000:01:00.0: success to request msi irq
                # 成功申请MSI 中断 log
[    2.280601] pcie-rkep 0000:01:00.0: successfully allocate continuouse buffer
for userspace   # 成功预留用户内存，默认未开启
[    2.280619] pcie-rkep 0000:01:00.0: vid=1d87
                # 正常访问EP卡config空间
[    2.280625] pcie-rkep 0000:01:00.0: did=356a
[    2.280629] pcie-rkep 0000:01:00.0: obj_info magic=524b4550, ver=100
                # 正常访问EP卡bar空间
```